

✓ Import Library yang digunakan

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.svm import SVC
6 from sklearn.model_selection import train_test_split
7 from sklearn.tree import DecisionTreeClassifier, plot_tree
8 from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, r2_
9 from sklearn.linear_model import LinearRegression
```

- `import pandas as pd`: Ini adalah tukang kebun data Anda. `pandas` sangat jago dalam mengurus data tabular (tabel atau spreadsheet), menjadikannya mudah untuk dibersihkan, diolah, dan dianalisis.
- `import numpy as np`: Ini adalah otak di balik layar. `numpy` menyediakan dukungan untuk array dan matriks besar, serta fungsi matematika tingkat tinggi. Hampir semua komputasi di Python yang melibatkan angka besar bergantung pada kecepatan `numpy`.
- `import matplotlib.pyplot as plt`: Kanvas dasar Anda. Digunakan untuk membuat plot dan grafik statis secara umum. Ini adalah fondasi visualisasi di Python.
- `import seaborn as sns`: Penata gaya untuk plot. Dibangun di atas `matplotlib`, `seaborn` membuat grafik statistik yang kompleks dan menarik (seperti heatmap atau distribusi data) menjadi mudah dibuat dengan sedikit kode.
- `from sklearn.model_selection import train_test_split`: Wasit yang membagi data. Fungsinya adalah memisahkan data Anda menjadi set pelatihan (training) dan pengujian (testing), agar Anda bisa menguji model secara jujur di data yang belum pernah dilihat sebelumnya.
- `from sklearn.svm import SVC`: Pembuat batas yang canggih. Support Vector Classifier (SVC) adalah algoritma klasifikasi kuat yang mencari 'garis pemisah' terbaik (disebut hyperplane) antar kelas data.
- `from sklearn.tree import DecisionTreeClassifier`: Pohon keputusan yang transparan. Algoritma klasifikasi yang bekerja seperti diagram alir. Mudah dipahami karena menunjukkan langkah-langkah logis yang diambilnya.
- `from sklearn.tree import plot_tree`: Juru gambar pohon. Fungsi khusus ini mengambil model Decision Tree Anda dan mengubahnya menjadi diagram visual yang bisa Anda lihat dan jelaskan.
- `from sklearn.linear_model import LinearRegression`: Pencari garis lurus. Algoritma regresi paling dasar yang mencari hubungan garis lurus terbaik antara variabel untuk memprediksi nilai kontinu (seperti harga atau suhu).
- `from sklearn.metrics import accuracy_score`: Metrik utama untuk akurasi model klasifikasi—seberapa sering model menebak dengan benar.
- `from sklearn.metrics import confusion_matrix`: Papan skor model. Ini adalah tabel yang menunjukkan di mana model Anda bingung (salah memprediksi) dan di mana ia benar.
- `from sklearn.metrics import classification_report`: Laporan ringkas yang memberikan metrik penting seperti Precision, Recall, dan F1-score, yang lebih detail daripada sekadar akurasi.

- from sklearn.metrics import r2_score: Pengukur kecocokan untuk regresi. R^2 (R-squared) menunjukkan seberapa baik prediksi model regresi Anda sesuai dengan data aktual.
- from sklearn.metrics import mean_squared_error: Pengukur kesalahan model regresi. Nilai rata-rata dari kuadrat perbedaan antara prediksi model dan nilai sebenarnya. Semakin kecil nilainya, semakin baik model Anda.

✓ Load Data

```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/c

from google.colab import drive untuk memanggil modul drive dari Google Colab.

drive.mount('/[content/drive](#)') untuk menyambungkan/menghubungkan Google Colab ke Google Drive

```
1 path = '/content/drive/MyDrive/MACHINELEARNING/PRAKTIKUM/PRAKTIKUM7/DATA/'
```

Perintah di atas untuk membuat alamat folder tersebut ke dalam variabel bernama path

```
1 df = pd.read_csv(path + "dataset_satelit.csv")
2 df
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.220
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.189
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.147
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.182
...	...	...	...	...	...	...	...	...	...	...	...	...	...
589	590	103.605867	0.057633	2.49	0.16	0.347	0.78	0.86	63.38	269.95	...	0.2336	0.130
590	591	103.606717	0.057100	2.74	0.15	0.466	0.73	0.5	51.04	683.42	...	0.2506	0.212
591	592	103.606250	0.056767	2.63	0.15	0.422	0.82	0.59	82.57	396.18	...	0.3413	0.277
592	593	103.606400	0.056517	2.75	0.17	0.502	0.69	0.53	102.07	246.35	...	0.3413	0.327
593	594	103.606033	0.056317	2.71	0.15	0.419	0.78	0.46	90.60	371.46	...	0.3482	0.412

594 rows × 34 columns

Perintah di atas untuk membaca file dengan format csv menggunakan library pandas

```
1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
#   Column      Non-Null Count  Dtype
```

```

0  No          594 non-null  int64
1  Longitude   594 non-null  float64
2  Lattitude   594 non-null  float64
3  N           594 non-null  float64
4  P           594 non-null  float64
5  K           593 non-null  float64
6  Ca          594 non-null  float64
7  Mg          594 non-null  object
8  Fe          594 non-null  float64
9  Mn          594 non-null  float64
10 Cu          594 non-null  float64
11 Zn          594 non-null  float64
12 B           594 non-null  float64
13 b12         594 non-null  float64
14 b11         594 non-null  float64
15 b9          594 non-null  float64
16 b8a         594 non-null  float64
17 b8          594 non-null  float64
18 b7          594 non-null  float64
19 b6          594 non-null  float64
20 b5          594 non-null  float64
21 b4          594 non-null  float64
22 b3          594 non-null  float64
23 b2          594 non-null  float64
24 b1          594 non-null  float64
25 Sigma_VV    594 non-null  float64
26 Sigma_VH    594 non-null  float64
27 plia        594 non-null  float64
28 lia         594 non-null  float64
29 iafe        594 non-null  float64
30 gamma0_vv   594 non-null  float64
31 gamma0_vh   594 non-null  float64
32 beta0_vv    594 non-null  float64
33 beta0_vh    594 non-null  float64
dtypes: float64(32), int64(1), object(1)
memory usage: 157.9+ KB

```

Perintah diatas untuk mengecek info dari dataset yang dibaca

- **No** : nomor urut data.
- **Longitude** : posisi garis bujur (arah timur & barat).
- **Lattitude** : posisi garis lintang (arah utara & selatan).
- **N, P, K** : kadar unsur hara utama tanah: Nitrogen, Fosfor, Kalium.
- **Ca, Mg, Fe, Mn, Cu, Zn, B, Mo, Na** : unsur kimia tanah lainnya (kalsium, magnesium, besi, mangan, tembaga, seng, boron, molibdenum, natrium).
- **pH_H2O** : tingkat keasaman tanah diukur dengan air.
- **pH_KCl** : tingkat keasaman tanah diukur dengan larutan KCl.
- **C_Org** : kandungan karbon organik tanah.
- **CEC** : kapasitas tukar kation, kemampuan tanah menahan unsur hara.
- **b1 – b7** : nilai reflektansi dari band citra satelit (misal Landsat/Sentinel).
- **Sigma_VV, Sigma_VH** : nilai pantulan radar satelit dengan polarisasi VV dan VH.
- **plia, lia, iafe** : parameter sudut datang atau pantulan sinyal radar. keyboard_arrow_down Interpretasi setiap kolom
- **gamma0_vv, gamma0_vh, beta0_vv, beta0_vh** : hasil koreksi nilai pantulan radar (untuk analisis permukaan tanah).

```
1 df.isnull().sum()
```

	θ
No	0
Longitude	0
Lattitude	0
N	0
P	0
K	1
Ca	0
Mg	0
Fe	0
Mn	0
Cu	0
Zn	0
B	0
b12	0
b11	0
b9	0
b8a	0
b8	0
b7	0
b6	0
b5	0
b4	0
b3	0
b2	0
b1	0
Sigma_VV	0
Sigma_VH	0
plia	0
lia	0
iafe	0
gamma0_vv	0
gamma0_vh	0
beta0_vv	0

beta0_vh 0

Perintah diatas untuk mengecek missing value yang ada di dataset

dtype: int64

```
1 df.describe()
```

	No	Longitude	Lattitude	N	P	K	Ca	
count	594.000000	594.000000	594.000000	594.000000	594.000000	593.000000	594.000000	594.00
mean	297.500000	106.878644	-1.024933	2.259091	0.141380	0.582175	0.595094	74.61
std	171.617307	4.949840	0.965349	0.395499	0.019782	0.222567	0.366118	55.57
min	1.000000	102.760857	-2.333750	1.140000	0.090000	0.122000	0.050000	21.08
25%	149.250000	102.927811	-2.233338	1.982500	0.130000	0.429000	0.320000	40.70
50%	297.500000	103.581969	-0.602276	2.280000	0.140000	0.549000	0.540000	65.65
75%	445.750000	113.403797	-0.257349	2.570000	0.150000	0.710000	0.790000	87.37
max	594.000000	113.434700	0.069251	3.230000	0.220000	1.489000	2.820000	559.10

8 rows × 33 columns

Perintah di atas untuk menampilkan statistik deskriptif dari kolom-kolom yang bertipe angka (numerik) dalam dataset

- **count** : jumlah data (baris) yang ada di tiap kolom
- **mean** : nilai rata-rata
- **std** : standar deviasi (tingkat sebaran data dari rata-ratanya)
- **min** : nilai terkecil
- **25%** : nilai pada persentil 25 (seperempat bagian bawah data)
- **50%** : nilai tengah (median)
- **75%** : nilai pada persentil 75 (tiga perempat bagian atas data)
- **max** : nilai terbesar

```
1 df.duplicated().sum()
```

```
np.int64(0)
```

```
1 df = df.drop_duplicates()
```

```
1 df.duplicated().sum()
```

```
np.int64(0)
```

Perintah diatas untuk mengecek data duplicat yang ada didataset dan mengdropnya atau menghapus

```
1 df.columns
```

```
Index(['No', 'Longitude', 'Lattitude', 'N', 'P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
      'Cu', 'Zn', 'B', 'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
      'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
      'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh'],
      dtype='object')
```

Perintah di atas untuk menampilkan daftar kolom pada dataset

```
1 df['Mg'] = pd.to_numeric(df['Mg'], errors='coerce')
2 df=df.dropna()
```

df = df.dropna() digunakan untuk menghapus semua baris yang berisi nilai kosong (NaN) dari dataset

Feature Selection

```
1 #X= df[['b2', 'b3', 'b4', 'b8', 'b11', 'Sigma_VV', 'Sigma_VH']]
2 #y= df['N']
```

Perintah di atas sengaja di ubah menjadi komentar karena ingin memakai variable lebih banyak

```
1 # feature selection
2 X = df[['P', 'K', 'Ca', 'Mg', 'Fe', 'Mn', 'Cu', 'Zn', 'B',
3         'b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5',
4         'b4', 'b3', 'b2', 'b1', 'Sigma_VV', 'Sigma_VH', 'plia', 'lia', 'iafe',
5         'gamma0_vv', 'gamma0_vh', 'beta0_vv', 'beta0_vh']]
6 y = df['N']
7
```

Perintah diatas untuk membagi fitur dan juga target

Splitting Data

```
1 #split data
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y, test_size=0.2, random_state=42
4 )
```

Mensplitting data atau membagi data menjadi 20% data testing dan 80% training

```
1 model = LinearRegression()
2 model.fit(X_train, y_train)
```

LinearRegression ⓘ ?

LinearRegression()

Perintah Datas untuk membuat model yang ingin digunakan yaitu linear regression

Machine

```
1 # testing
2 y_pred = model.predict(X_test)
3 # evaluasi model
4 r2 = r2_score(y_test, y_pred)
5 mse = mean_squared_error(y_test, y_pred)
6 rmse = np.sqrt(mse)
7 print("R² squared:", r2)
8 print("RMSE:", rmse)
9
```

R² squared: 0.7549955124484078
RMSE: 0.19392193397227142

Perintah diatas untuk mengetahui sebaik apa model regresi memprediksi

```

1 coeff = pd.DataFrame({
2   'Fitur': X_train.columns,
3   'Koefisien': model.coef_
4 })
5 print(coeff)

```

	Fitur	Koefisien
0	P	8.570837
1	K	0.056014
2	Ca	-0.058380
3	Mg	-0.161095
4	Fe	0.000284
5	Mn	0.000177
6	Cu	0.023202
7	Zn	0.002968
8	B	-0.002092
9	b12	0.327312
10	b11	-0.619437
11	b9	-0.047586
12	b8a	0.064453
13	b8	-0.072659
14	b7	0.981186
15	b6	-0.864947
16	b5	-0.329397
17	b4	1.420614
18	b3	-0.449043
19	b2	-0.357454
20	b1	-0.033418
21	Sigma_VV	0.156595
22	Sigma_VH	-1.466200
23	plia	0.021315
24	lia	-0.021906
25	iafe	-0.078122
26	gamma0_vv	0.457327
27	gamma0_vh	2.946416
28	beta0_vv	-0.569281
29	beta0_vh	-0.677599

Perintah diatas untuk mengetahui tabel fiturr

```

1 import statsmodels.api as sm
2 X_sm = sm.add_constant(X)
3
4 # tambahkan intercept
5 model_ols = sm.OLS(y, X_sm).fit()
6 print(model_ols.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          N      R-squared:          0.779
Model:                  OLS    Adj. R-squared:        0.768
Method:                 Least Squares    F-statistic:          66.10
Date:                  Fri, 07 Nov 2025    Prob (F-statistic):    6.69e-163
Time:                  15:41:21    Log-Likelihood:       156.68
No. Observations:      592    AIC:                  -251.4
Df Residuals:          561    BIC:                  -115.5
Df Model:              30
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	3.8681	0.864	4.479	0.000	2.172	5.564
P	8.5760	0.465	18.428	0.000	7.662	9.490
K	0.0273	0.051	0.530	0.596	-0.074	0.128
Ca	-0.0416	0.023	-1.808	0.071	-0.087	0.004
Mg	-0.2166	0.063	-3.464	0.001	-0.339	-0.094

```

Fe          0.0002    0.000    1.102    0.271    -0.000    0.001
Mn          0.0002    4.59e-05    4.083    0.000    9.72e-05    0.000
Cu          0.0229    0.006    4.046    0.000    0.012    0.034
Zn          0.0024    0.001    1.889    0.059    -9.52e-05    0.005
B           -0.0022    0.001    -1.476    0.141    -0.005    0.001
b12         0.2010    0.534    0.377    0.707    -0.848    1.250
b11         -0.9945    0.517    -1.922    0.055    -2.011    0.022
b9          -0.0802    0.067    -1.191    0.234    -0.212    0.052
b8a         0.0962    0.067    1.427    0.154    -0.036    0.229
b8          -0.0806    0.024    -3.362    0.001    -0.128    -0.034
b7          1.3059    0.542    2.409    0.016    0.241    2.371
b6          -0.9238    0.462    -2.001    0.046    -1.831    -0.017
b5          -0.5470    0.488    -1.120    0.263    -1.506    0.412
b4          1.2561    0.972    1.292    0.197    -0.653    3.166
b3          0.3085    1.639    0.188    0.851    -2.910    3.527
b2          -0.7075    0.942    -0.751    0.453    -2.559    1.144
b1          0.0023    0.352    0.006    0.995    -0.689    0.694
Sigma_VV    0.9543    0.889    1.074    0.283    -0.792    2.700
Sigma_VH    -1.1666    0.861    -1.356    0.176    -2.857    0.524
plia        -0.0113    0.071    -0.159    0.873    -0.150    0.128
lia         0.0102    0.071    0.144    0.886    -0.129    0.149
iafe        -0.0802    0.023    -3.475    0.001    -0.126    -0.035
gamma0_vv   -0.5040    2.180    -0.231    0.817    -4.785    3.777
gamma0_vh    3.1827    4.643    0.685    0.493    -5.938    12.303
beta0_vv    -0.2381    1.580    -0.151    0.880    -3.342    2.865
beta0_vh    -0.5472    3.536    -0.155    0.877    -7.493    6.399
=====
Omnibus:                28.388    Durbin-Watson:                1.527
Prob(Omnibus):           0.000    Jarque-Bera (JB):             53.216
Skew:                    -0.310    Prob(JB):                     2.78e-12
Kurtosis:                4.332    Cond. No.                     2.98e+05
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 2.98e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Perintah diatas membuat analisis regresi linear dengan metode OLS